

Etapa III: Optimización y Rendimiento



Grupo Número 5

Alumnos:

- Yapura Fuenzalida Víctor - M.U.N: 01913
- Escalante Judith Griselda - M.U.N: 01862
- Moreira Joaquín - M.U.N: 01884
- Quintero Facundo Joel - M.U.N: 01847
- Gallo Juan Agustín - M.U.N: 01731
- Bravo Nicolas - M.U.N: 01805

Carrera: Ingeniería en Informática

Año: 2026

Profesores:

- Bustos Aguiar Soledad
- Mgtr. Manuel Baquinzay
- Juan Matias Reynoso Chocobar



Introducción y Justificación Técnica de la Indexación

En el diseño de bases de datos relacionales, el motor genera de manera automática estructuras de búsqueda rápida para las claves primarias (PRIMARY KEY). Sin embargo, deja las claves foráneas (FOREIGN KEY) completamente desprovistas de mecanismos de aceleración físicos.

Cuando el sistema ejecuta consultas avanzadas que cruzan información mediante operaciones de unión (JOIN), el motor localiza la clave primaria de forma instantánea, pero al intentar emparejarla con su contraparte foránea, se ve obligado a realizar un escaneo secuencial (Seq Scan). Esto significa que debe leer la tabla completa en el disco duro, registro por registro. Con grandes volúmenes de datos, este comportamiento satura los recursos del servidor, eleva los tiempos de respuesta y degrada la escalabilidad del sistema en producción.

Para solucionar esta ineficiencia sin alterar la lógica de las consultas, se implementa una **indexación estratégica mediante estructuras de árbol balanceado (B-Tree)** sobre las claves foráneas, logrando las siguientes mejoras estructurales:

- **Evita lecturas totales en disco:** El motor consulta el índice como una bitácora de punteros ordenados y salta directamente al bloque físico donde reside el dato buscado, omitiendo registros irrelevantes.
- **Optimiza el plan de ejecución:** Transforma los costosos Seq Scan en accesos directos por puntero de tipo Index Scan, liberando memoria RAM y ciclos de CPU.
- **Garantiza la escalabilidad productiva:** Al reducir el costo de las operaciones de filtrado y acoplamiento, los tiempos de procesamiento se desploman de segundos a milisegundos, permitiendo soportar cargas de estrés masivas.

TABLA: DETALLE_MOVIMIENTOS

FASE 1: Estado Inicial (EI ANTES)

- **Métrica de Rendimiento:** Execution Time: 8.809 ms.
- **Diagnóstico del Query Plan:** El planificador arrojó la directiva Seq Scan on detalle_movimientos.
- **Causa de la Demora:** Al no indexarse automáticamente las claves foráneas, el motor realiza un escaneo secuencial leyendo la tabla completa de arriba a abajo en disco, ralentizando la respuesta.

```
20 -- Prueba de rendimiento de la tabla "Detalle_Movimientos"
21 EXPLAIN ANALYZE
22 SELECT
23     mrc.nombre AS marca_deportiva,
24     SUM(dm.cantidad) AS total_prendas_vendidas,
25     SUM(dm.cantidad * dm.precio_unitario) AS recaudacion_total
26 FROM Movimientos mov
27 INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
28 INNER JOIN Producto_Variante pv ON dm.id_variante = pv.id_variante
29 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
30 INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
31 WHERE mov.tipo_movimiento = 'Salida'
32 AND mov.fecha_hora >= CURRENT_DATE - INTERVAL '3 months'
33 GROUP BY mrc.id_marca, mrc.nombre
34 ORDER BY recaudacion_total DESC;
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 44 Page No: 1 of 1

QUERY PLAN

text

7 Buffers: shared hit=2 read=9

8 -> Index Scan using producto_variante_pkey on producto_variante pv (cost=0.15..0.18 rows=1 width=8) (actual time=0.006..0.006 rows=1.00 loops=1)

9 Index Cond: (id_variante = dm.id_variante)

10 Index Searches: 44

11 Buffers: shared hit=86 read=2

12 -> Index Scan using productos_pkey on productos prod (cost=0.14..0.26 rows=1 width=8) (actual time=0.005..0.005 rows=1.00 loops=44)

13 Index Cond: (id_producto = pv.id_producto)

14 Index Searches: 44

15 Buffers: shared hit=86 read=2

16 -> Hash (cost=13.20..13.20 rows=320 width=222) (actual time=2.218..2.219 rows=4.00 loops=1)

17 Buckets: 1024 Batches: 1 Memory Usage: 9kB

18 Buffers: shared read=1 dirtied=1

19 -> Seq Scan on marcas mrc (cost=0.00..13.20 rows=320 width=222) (actual time=2.202..2.204 rows=4.00 loops=1)

20 Buffers: shared read=1 dirtied=1

21 Planning:

22 Buffers: shared hit=226 read=22 dirtied=1

23 Planning Time: 19.873 ms

24 Execution Time: 8.809 ms

FASE 2: Implementación Física (La Solución)

Se ejecutó el script de creación de índices B-Tree sobre las claves foráneas de la tabla para proveer caminos de acceso directo en las uniones:

```
25 ORDER BY recaudacion_total DESC;
```

```
26
```

```
27 -- Creacion de indices para optimizar el rendimiento de las consultas
```

```
28 -- Índice para acelerar los JOINS con la tabla Movimientos (Query 1, 3 y 5)
```

```
29 CREATE INDEX idx_detalle_movimientos_id_mov
```

```
30 ON public.Detalle_Movimientos(id_movimiento);
```

```
31
```

```
32 -- Índice para acelerar los JOINS con las variantes de producto (Query 2)
```

```
33 CREATE INDEX idx_detalle_movimientos_id_var
```

```
34 ON public.Detalle_Movimientos(id_variante);
```

Data Output Mensajes Notificaciones

CREATE INDEX

Consulta retornó exitosamente en 202 msec.

FASE 3: Estado Optimizado (El DESPUÉS)

- **Métrica de Rendimiento:** Execution Time: 2.403 ms.
- **Diagnóstico del Query Plan:** El planificador sustituyó el barrido total por accesos directos por puntero de tipo **Index Scan** y **Bitmap Heap Scan**.
- **Análisis del Resultado:** La consulta redujo drásticamente su tiempo de procesamiento. El comando residual Seq Scan on marcas mrc es correcto y óptimo, debido a que la tabla posee únicamente 4 filas y el motor determina que es más

eficiente barrerlas directamente en memoria RAM que abrir la infraestructura del índice.

```

11 -- Prueba de rendimiento de la tabla "Detalle_Movimientos"
12 EXPLAIN ANALYZE
13 SELECT
14     mrc.nombre AS marca_deportiva,
15     SUM(dm.cantidad) AS total_prendas_vendidas,
16     SUM(dm.cantidad * dm.precio_unitario) AS recaudacion_total
17 FROM Movimientos mov
18 INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
19 INNER JOIN Producto_Variante pv ON dm.id_variante = pv.id_variante
20 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
21 INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
22 WHERE mov.tipo_movimiento = 'Salida'
23 AND mov.fecha_hora >= CURRENT_DATE - INTERVAL '3 months'
24 GROUP BY mrc.id_marca, mrc.nombre
25 ORDER BY recaudacion_total DESC;

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 44 Page No:

QUERY PLAN	text
26	rows removed by filter: 961
27	Buffers: shared hit=11
28	-> Index Scan using producto_variante_pkey on producto_variante pv (cost=0.15..0.18 rows=1 width=8) (actual time=0.001..0.001 rows=1.00 loops=1)
29	Index Cond: (id_variante = dm.id_variante)
30	Index Searches: 44
31	Buffers: shared hit=88
32	-> Index Scan using productos_pkey on productos prod (cost=0.14..0.26 rows=1 width=8) (actual time=0.002..0.002 rows=1.00 loops=44)
33	Index Cond: (id_producto = pv.id_producto)
34	Index Searches: 44
35	Buffers: shared hit=88
36	-> Hash (cost=13.20..13.20 rows=320 width=222) (actual time=0.262..0.262 rows=4.00 loops=1)
37	Buckets: 1024 Batches: 1 Memory Usage: 9kB
38	Buffers: shared hit=1
39	-> Seq Scan on marcas mrc (cost=0.00..13.20 rows=320 width=222) (actual time=0.049..0.051 rows=4.00 loops=1)
40	Buffers: shared hit=1
41	Planning:
42	Buffers: shared hit=41 read=5
43	Planning Time: 7.172 ms
44	Execution Time: 2.403 ms

TABLA 2: MOVIMIENTOS

Esta entidad transaccional registra de forma cronológica todas las operaciones físicas de entrada, salida y traslado de mercadería dentro del ecosistema. Para auditar y optimizar su comportamiento físico, se tomó como caso de estudio la **Consulta Avanzada N° 3 (Reporte de Rendimiento de Empleados en Sucursal NOA)**

FASE 1: Estado Inicial (El ANTES)

- **Métrica de Rendimiento:** Execution Time: 1.662 ms.
- **Diagnóstico del Query Plan:** El planificador arrojó la directiva Seq Scan on movimientos mov.
- **Causa de la Demora:** Al carecer de índices en sus claves foráneas, el motor recorre secuencialmente todo el historial de movimientos para enlazar los empleados y las sucursales.

```
38 -- Prueba de rendimiento de la tabla "Movimientos"
39 EXPLAIN ANALYZE
40 SELECT
41     emp.nombre,
42     emp.apellido,
43     suc.nombre AS sucursal,
44     COUNT(DISTINCT mov.id_movimiento) AS cantidad_tickets_emitidos,
45     SUM(dm.cantidad * dm.precio_unitario) AS total_plata_ingresada
46 FROM Movimientos mov
47 INNER JOIN Empleados emp ON mov.id_empleado = emp.id_empleado
48 INNER JOIN Sucursales suc ON mov.id_sucursal_origen = suc.id_sucursal
49 INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
50 WHERE mov.tipo_movimiento = 'Salida'
51     AND suc.nombre = 'Sucursal NOA'
52     AND EXTRACT(YEAR FROM mov.fecha_hora) = 2025
53 GROUP BY
54     emp.id_empleado,
55     emp.nombre,
56     emp.apellido,
57     suc.nombre
58 ORDER BY
59     cantidad_tickets_emitidos DESC
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 39 Page No: 1 of 1

QUERY PLAN
text
rows removed by filter: 0/3
Buffers: shared hit=11
-> Index Scan using empleados_pkey on empleados emp (cost=0.14..8.16 rows=1 width=440) (actual time=0.001..0.001 rows=1.00 loops=153)
Index Cond: (id_empleado = mov.id_empleado)
Index Searches: 153
Buffers: shared hit=306
-> Index Scan using idx_detalle_movimientos_id_mov on detalle_movimientos dm (cost=0.28..8.31 rows=2 width=16) (actual time=0.001..0.002 rows=2.41 loop=...
Index Cond: (id_movimiento = mov.id_movimiento)
Index Searches: 153
Buffers: shared hit=460
Planning:
Buffers: shared hit=12
Planning Time: 0.981 ms
Execution Time: 1.662 ms

FASE 2: Implementación Física (La Solución)

Se aplicó el script de creación de índices B-Tree sobre las claves foráneas de relación (id_empleado e id_sucursal_origen) para acelerar los acoplamientos lógicos en los cruzamientos de datos:

```
62 -- Creación de índices para optimizar el rendimiento de las consultas
63 -- 1. Índice estratégico para acelerar la vinculación con la tabla de Empleados
64 CREATE INDEX idx_movimientos_id_empleado
65 ON public.Movimientos(id_empleado);
66
67 -- 2. Índice estratégico para acelerar el filtrado y JOIN con la tabla de Sucursales
68 CREATE INDEX idx_movimientos_id_sucursal_origen
69 ON public.Movimientos(id_sucursal_origen);
```

Data Output Mensajes Notificaciones

CREATE INDEX

Consulta retornó exitosamente en 106 msec.

FASE 3: Estado Optimizado (El DESPUÉS)

- **Métrica de Rendimiento:** Execution Time: 1.282 ms.
- **Diagnóstico del Query Plan:** El planificador sustituyó el rastreo lineal de registros por accesos directos por puntero de tipo **Index Scan** y combinaciones indexadas.
- **Análisis del resultado:** La optimización redujo el tiempo de procesamiento de la consulta de forma drástica. El motor ahora localiza las operaciones de la sucursal y

el empleado emisor directamente desde las estructuras B-Tree en memoria RAM, evitando escaneos costosos y asegurando la velocidad del reporte.

```

38 -- Prueba de rendimiento de la tabla "Movimientos"
39 EXPLAIN ANALYZE
40 SELECT
41     emp.nombre,
42     emp.apellido,
43     suc.nombre AS sucursal,
44     COUNT(DISTINCT mov.id_movimiento) AS cantidad_tickets_emitidos,
45     SUM(dm.cantidad * dm.precio_unitario) AS total_plata_ingresada
46 FROM Movimientos mov
47 INNER JOIN Empleados emp ON mov.id_empleado = emp.id_empleado
48 INNER JOIN Sucursales suc ON mov.id_sucursal_origen = suc.id_sucursal
49 INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
50 WHERE mov.tipo_movimiento = 'Salida'
51     AND suc.nombre = 'Sucursal NOA'
52     AND EXTRACT(YEAR FROM mov.fecha_hora) = 2025
53 GROUP BY
54     emp.id_empleado,
55     emp.nombre,
56     emp.apellido,
57     suc.nombre
58 ORDER BY
59     cantidad_tickets_emitidos DESC,
60     emp.nombre,
61     emp.apellido,
62     suc.nombre

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 39 | Page No: 1 of 1

QUERY PLAN	text
27	Buffers: shared hit=11
28	→ Index Scan using empleados_pkey on empleados emp (cost=0.14..8.16 rows=1 width=440) (actual time=0.001..0.001 rows=1 loops=153)
29	Index Cond: (id_empleado = mov.id_empleado)
30	Index Searches: 153
31	Buffers: shared hit=306
32	→ Index Scan using idx_detalle_movimientos_id_mov on detalle_movimientos dm (cost=0.28..8.31 rows=2 width=16) (actual time=0.001..0.002 rows=2.41 loop_...)
33	Index Cond: (id_movimiento = mov.id_movimiento)
34	Index Searches: 153
35	Buffers: shared hit=460
36	Planning:
37	Buffers: shared hit=43 read=2
38	Planning Time: 3.551 ms
39	Execution Time: 1.282 ms

TABLA 3: INVENTARIO

Esta entidad centraliza el control de las existencias físicas disponibles para cada una de las variantes de ropa deportiva en las distintas sucursales. Para auditar y optimizar su comportamiento bajo consultas de control operativo, se tomó como caso de estudio la **Consulta Avanzada N° 4 (Reporte de Alertas de Bajo Stock)**:

FASE 1: Estado Inicial (El ANTES)

- **Métrica de Rendimiento:** Execution Time: 2.386 ms.
- **Diagnóstico del Query Plan:** El planificador ejecuta un escaneo secuencial completo sobre la tabla para evaluar el filtro condicional del volumen disponible.
- **Causa de la Demora:** *Al no contar con un índice en la columna de existencias, el motor barre secuencialmente toda la tabla analizando cada registro de stock de forma lineal.*

```
-- Prueba de rendimiento de la tabla "Inventario"
EXPLAIN ANALYZE
SELECT
  suc.nombre AS sucursal,
  prod.nombre AS producto,
  tal.nomenclatura AS talla,
  col.nombre_color AS color,
  inv.cantidad_disponible AS stock_actual
FROM Inventario inv
INNER JOIN Sucursales suc ON inv.id_sucursal = suc.id_sucursal
INNER JOIN Producto_Variante pv ON inv.id_variante = pv.id_variante
INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
INNER JOIN Talles tal ON pv.id_talle = tal.id_talle
INNER JOIN Colores col ON pv.id_color = col.id_color
WHERE inv.cantidad_disponible < 35
ORDER BY
  suc.nombre ASC;
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 42 Page No: 1 of 1

QUERY PLAN

text

26 Buffers: shared hit=4

27 -> Index Scan using productos_pkey on productos prod (cost=0.14..0.26 rows=1 width=322) (actual time=0.007..0.007 rows=1.00 loops=2)

28 Index Cond: (id_producto = pv.id_producto)

29 Index Searches: 2

30 Buffers: shared hit=4

31 -> Index Scan using talles_pkey on talles tal (cost=0.15..0.26 rows=1 width=62) (actual time=0.130..0.130 rows=1.00 loops=2)

32 Index Cond: (id_talle = pv.id_talle)

33 Index Searches: 2

34 Buffers: shared hit=2 read=2 dirtied=1

35 -> Index Scan using colores_pkey on colores col (cost=0.15..0.26 rows=1 width=122) (actual time=0.110..0.110 rows=1.00 loops=2)

36 Index Cond: (id_color = pv.id_color)

37 Index Searches: 2

38 Buffers: shared hit=2 read=2 dirtied=1

39 Planning:

40 Buffers: shared hit=107 read=6 dirtied=1

41 Planning Time: 10.389 ms

42 Execution Time: 2.386 ms

FASE 2: Implementación Física (La Solución)

Se inyectó en el catálogo un índice B-Tree diseñado específicamente para dar soporte rápido a las búsquedas por rangos y filtros de corte numérico de la columna condicional:

```
-- Creacion de indices para optimizar el rendimiento de las consultas
CREATE INDEX idx_inventario_cantidad_disponible
ON public.Inventario(cantidad_disponible);
```

Data Output Mensajes Notificaciones

CREATE INDEX

Consulta retornó exitosamente en 88 msec.

FASE 3: Estado Optimizado (El DESPUÉS)

- **Métrica de Rendimiento:** Execution Time: 0.132 ms.
- **Diagnóstico del Query Plan:** El planificador sustituye el escaneo lineal de registros por una consulta directa por puntero estructurado de tipo **Index Scan**.
- **Análisis del Resultado:** La creación del índice específico redujo el tiempo de procesamiento de 2.386 ms a un valor óptimo de 0.132 ms. Al consolidar la localización de saldos críticos de stock directamente en las páginas de la memoria

intermedia, el motor elimina la redundancia de procesamiento y blinda la escalabilidad del reporte de reposición.

```

73 -- Prueba de rendimiento de la tabla "Inventario"
74 EXPLAIN ANALYZE
75 SELECT
76     suc.nombre AS sucursal,
77     prod.nombre AS producto,
78     tal.nomenclatura AS talla,
79     col.nombre_color AS color,
80     inv.cantidad_disponible AS stock_actual
81 FROM Inventario inv
82 INNER JOIN Sucursales suc ON inv.id_sucursal = suc.id_sucursal
83 INNER JOIN Producto_Variante pv ON inv.id_variante = pv.id_variante
84 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
85 INNER JOIN Tallas tal ON pv.id_talla = tal.id_talla
86 INNER JOIN Colores col ON pv.id_color = col.id_color
87 WHERE inv.cantidad_disponible < 35
88 ORDER BY
89     suc.nombre ASC,
90     suc.cantidad_disponible DESC;

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 42 Page No: 1 of 1

QUERY PLAN	text
26	Buffers: shared hit=4
27	-> Index Scan using productos_pkey on productos prod (cost=0.14..0.26 rows=1 width=322) (actual time=0.006..0.006 rows=1.00 loops=2)
28	Index Cond: (id_producto = pv.id_producto)
29	Index Searches: 2
30	Buffers: shared hit=4
31	-> Index Scan using tallas_pkey on tallas tal (cost=0.15..0.26 rows=1 width=62) (actual time=0.003..0.003 rows=1.00 loops=2)
32	Index Cond: (id_talla = pv.id_talla)
33	Index Searches: 2
34	Buffers: shared hit=4
35	-> Index Scan using colores_pkey on colores col (cost=0.15..0.26 rows=1 width=122) (actual time=0.003..0.003 rows=1.00 loops=2)
36	Index Cond: (id_color = pv.id_color)
37	Index Searches: 2
38	Buffers: shared hit=4
39	Planning:
40	Buffers: shared hit=21 read=1
41	Planning Time: 1.951 ms
42	Execution Time: 0.132 ms

TABLA 4: PRODUCTO_VARIANTE

Esta entidad catálogo actúa como el puente relacional que unifica cada producto con sus correspondientes tallas y colores en el sistema. Para auditar su comportamiento y la eficiencia de los cruzamientos lógicos, se tomó como caso de estudio la **Consulta Avanzada N° 2 (Reporte de Variantes sin Salida en el Año Actual)**, la cual procesa una subconsulta compleja con la cláusula condicional NOT IN.

FASE 1: Estado Inicial (El ANTES)

- **Métrica de Rendimiento:** Execution Time: 4.328 ms.
- **Diagnóstico del Query Plan:** El planificador registra múltiples operaciones de escaneo secuencial (Seq Scan) en las tablas de atributos asociadas durante la ejecución de los acoplamientos.
- **Causa de la Demora:** La ausencia de índices en claves foráneas obliga al motor a escanear secuencialmente las relaciones de atributos para resolver los cruzamientos de la subconsulta.


```
98 -- Prueba de rendimiento de la tabla "Producto_Variante"
99
100 EXPLAIN ANALYZE
101 SELECT
102     mrc.nombre AS marca,
103     prod.nombre AS producto,
104     tal.nomenclatura AS talla,
105     col.nombre_color AS color
106 FROM Producto_Variante pv
107 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
108 INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
109 INNER JOIN Talles tal ON pv.id_talle = tal.id_talle
110 INNER JOIN Colores col ON pv.id_color = col.id_color
111 WHERE pv.id_variante NOT IN (
112     SELECT dm.id_variante
113     FROM Detalle_Movimientos dm
114     INNER JOIN Movimientos mov ON dm.id_movimiento = mov.id_movimiento
115     WHERE mov.tipo_movimiento = 'Salida'
116     AND EXTRACT(YEAR FROM mov.fecha_hora) = EXTRACT(YEAR FROM CURRENT_DATE)
117 )
118 ORDER BY mrc.nombre, prod.nombre;
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 55 Page No: 1 of 1

QUERY PLAN

text

42 -> Hash (cost=19.00..19.00 rows=900 width=62) (actual time=0.033..0.033 rows=6.00 loops=1)

43 Buckets: 1024 Batches: 1 Memory Usage: 9kB

44 Buffers: shared hit=1 dirtied=1

45 -> Seq Scan on talles tal (cost=0.00..19.00 rows=900 width=62) (actual time=0.029..0.029 rows=6.00 loops=1)

46 Buffers: shared hit=1 dirtied=1

47 -> Hash (cost=15.40..15.40 rows=540 width=122) (actual time=0.517..0.518 rows=4.00 loops=1)

48 Buckets: 1024 Batches: 1 Memory Usage: 9kB

49 Buffers: shared hit=1 dirtied=1

50 -> Seq Scan on colores col (cost=0.00..15.40 rows=540 width=122) (actual time=0.282..0.284 rows=4.00 loops=1)

51 Buffers: shared hit=1 dirtied=1

52 Planning:

53 Buffers: shared hit=12

54 Planning Time: 5.063 ms

55 Execution Time: 4.328 ms

FASE 2: Implementación Física (La Solución)

Se subsanaron las deficiencias físicas mediante la creación de índices B-Tree sobre las tres claves foráneas de la entidad, eliminando errores de doble empaquetado de esquemas lógicos detectados en las pruebas de consola:

```
119
120 -- Creacion de índices para optimizar el rendimiento de las consultas
121 -- 1. Índice para acelerar el JOIN con la tabla de Productos
122 CREATE INDEX idx_producto_variante_id_producto
123 ON public.Producto_Variante(id_producto);
124
125 -- 2. Índice para acelerar el JOIN con la tabla de Talles
126 CREATE INDEX idx_producto_variante_id_talle
127 ON public.Producto_Variante(id_talle);
128
129 -- 3. Índice para acelerar el JOIN con la tabla de Colores
130 CREATE INDEX idx_producto_variante_id_color
131 ON public.Producto_Variante(id_color);
```

Data Output Mensajes Notificaciones

ERROR: cross-database references are not implemented: "public.public.producto_variante"

Estado SQL: 0A000

FASE 3: Estado Optimizado (El DESPUÉS)

- **Métrica de Rendimiento:** Execution Time: 0.909 ms.

- **Diagnóstico del Query Plan:** El planificador reestructuró la estrategia física de uniones, integrando búsquedas rápidas por puntero indexado de tipo **Index Scan**.
- **Análisis del Resultado:** El tiempo de procesamiento global se redujo drásticamente de 4.328 ms a un valor óptimo de 0.909 ms. Al indexar los vectores de atributos de las variantes, el motor evalúa la subconsulta de exclusión directamente en los búferes de memoria RAM. Los escaneos secuenciales residuales observados en los catálogos de talles y colores son correctos, dado que son tablas microscópicas y Postgres determina que es más eficiente barrerlas linealmente que inicializar la estructura del índice.

```

98 -- Prueba de rendimiento de la Tabla "Producto_Variante"
99
100 EXPLAIN ANALYZE
101 SELECT
102     mrc.nombre AS marca,
103     prod.nombre AS producto,
104     tal.nomenclatura AS talla,
105     col.nombre_color AS color
106 FROM Producto_Variante pv
107 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
108 INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
109 INNER JOIN Talles tal ON pv.id_talle = tal.id_talle
110 INNER JOIN Colores col ON pv.id_color = col.id_color
111 WHERE pv.id_variante NOT IN (
112     SELECT dm.id_variante
113     FROM Detalle_Movimientos dm
114     INNER JOIN Movimientos mov ON dm.id_movimiento = mov.id_movimiento
115     WHERE mov.tipo_movimiento = 'Salida'
116     AND EXTRACT(YEAR FROM mov.fecha_hora) = EXTRACT(YEAR FROM CURRENT_DATE)
117 )

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 53 Page No: 1 of 1

QUERY PLAN	
Line#	Text
39	Filter: (((tipo_movimiento)::text = 'Salida'::text) AND (EXTRACT(year FROM fecha_hora) = EXTRACT(year FROM CURRENT_DATE)))
40	Rows Removed by Filter: 879
41	Buffers: shared hit=11
42	-> Index Scan using idx_detalle_movimientos_id_mov on detalle_movimientos dm (cost=0.28..8.31 rows=2 width=8) (actual time=0.001..0.001)
43	Index Cond: (id_movimiento = mov.id_movimiento)
44	Index Searches: 121
45	Buffers: shared hit=364
46	-> Index Scan using marcas_pkey on marcas mrc (cost=0.15..0.41 rows=1 width=222) (actual time=0.127..0.127 rows=1 loops=1)
47	Index Cond: (id_marca = prod.id_marca)
48	Index Searches: 1
49	Buffers: shared hit=1 read=1
50	Planning:
51	Buffers: shared hit=22
52	Planning Time: 1.925 ms
53	Execution Time: 0.909 ms

TABLA 5: PRODUCTOS

Esta entidad actúa como el catálogo maestro donde se centraliza la información base de toda la indumentaria deportiva, vinculando cada artículo con su respectiva marca y categoría. Para auditar su comportamiento y eficiencia, se tomó como caso de estudio la **Consulta Avanzada N° 5 (Reporte Comparativo de Ventas y Recaudación por Marca en cada Sucursal)**:

FASE 1: Estado Inicial (El ANTES)

- **Métrica de Rendimiento:** Execution Time: 3.615 ms.
- **Diagnóstico del Query Plan:** El planificador registró la directiva Seq Scan on productos prod con un costo estimado alto (cost=0.00..12.00 rows=200).

- **Causa de la Demora:** Al no estar indexada la clave foránea de marcas, el motor realiza un escaneo secuencial evaluando todas las filas estimadas de la tabla productos.

```

135 -- Prueba de rendimiento de la tabla "Productos"
136 EXPLAIN ANALYZE
137 SELECT
138     suc.nombre AS sucursal,
139     mrc.nombre AS marca,
140     SUM(dm.cantidad) AS total_unidades_vendidas,
141     SUM(dm.cantidad * dm.precio_unitario) AS recaudacion_por_marca
142 FROM Movimientos mov
143 INNER JOIN Sucursales suc ON mov.id_sucursal_origen = suc.id_sucursal
144 INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
145 INNER JOIN Producto_Variante pv ON dm.id_variante = pv.id_variante
146 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
147 INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
148 WHERE mov.tipo_movimiento = 'Salida'
149 GROUP BY
150     suc.nombre,
151     mrc.nombre
152 ORDER BY
153     suc.nombre ASC,
154     recaudacion_por_marca DESC

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 56 Page No: 1 of 1

QUERY PLAN	Text
42	Buffers: shared hit-1
43	→ Hash (cost=12.00..12.00 rows=200 width=8) (actual time=0.010..0.011 rows=4.00 loops=1)
44	Buckets: 1024 Batches: 1 Memory Usage: 9kB
45	Buffers: shared hit-1
46	→ Seq Scan on productos prod (cost=0.00..12.00 rows=200 width=8) (actual time=0.007..0.008 rows=4.00 loops=1)
47	Buffers: shared hit-1
48	→ Hash (cost=13.20..13.20 rows=320 width=222) (actual time=0.036..0.037 rows=4.00 loops=1)
49	Buckets: 1024 Batches: 1 Memory Usage: 9kB
50	Buffers: shared hit-1
51	→ Seq Scan on marcas mrc (cost=0.00..13.20 rows=320 width=222) (actual time=0.030..0.031 rows=4.00 loops=1)
52	Buffers: shared hit-1
53	Planning:
54	Buffers: shared hit-12
55	Planning Time: 1.018 ms
56	Execution Time: 3.615 ms

Ejecución exitosa. Tiempo de ejecución total de la consulta: 125 msec. 56 filas afectadas.

FASE 2: Implementación Física (La Solución)

Se aplicó el script de creación de índices B-Tree sobre las claves foráneas de relación (id_marca e id_categoria) para acelerar los cruzamientos relacionales lógicos de los catálogos:

```

156 -- Creacion de indices para optimizar el rendimiento de las consultas
157 -- 1. Índice para acelerar el JOIN entre Productos y Marcas (Query 1, 2 y 5)
158 CREATE INDEX idx_productos_id_marca
159 ON public.Productos(id_marca);
160
161 -- 2. Índice estratégico para optimizar búsquedas y filtrados por Categoría
162 CREATE INDEX idx_productos_id_categoria
163 ON public.Productos(id_categoria);

```

Data Output Mensajes Notificaciones

CREATE INDEX

Consulta retornó exitosamente en 221 msec.

FASE 3: Estado Optimizado (El DESPUÉS)

- **Métrica de Rendimiento:** Execution Time: 2.316 ms.

- **Diagnóstico del Query Plan:** El planificador recalculó y redujo drásticamente el costo operativo de la consulta (cost=0.00..1.04 rows=4).
- **Análisis del Resultado:** El tiempo de procesamiento final se optimizó a 2.316 ms. El comando residual Seq Scan on productos prod es correcto y esperado por el motor, debido a que la tabla cuenta únicamente con 4 registros reales insertados en el entorno de pruebas. El optimizador determina matemáticamente que es más rápido barrer las 4 filas directamente en la memoria caché RAM que inicializar la lectura física de la estructura del índice.

```

134 -- Prueba de rendimiento de la tabla "Productos"
135 EXPLAIN ANALYZE
136 SELECT
137     suc.nombre AS sucursal,
138     mrc.nombre AS marca,
139     SUM(dm.cantidad) AS total_unidades_vendidas,
140     SUM(dm.cantidad * dm.precio_unitario) AS recaudacion_por_marca
141 FROM Movimientos mov
142 INNER JOIN Sucursales suc ON mov.id_sucursal_origen = suc.id_sucursal
143 INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
144 INNER JOIN Producto_Variante pv ON dm.id_variante = pv.id_variante
145 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
146 INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
147 WHERE mov.tipo_movimiento = 'Salida'
148 GROUP BY
149     suc.nombre,
150     mrc.nombre
151 ORDER BY
152     suc.nombre ASC,
153

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 56 Page No: 1 of 1

QUERY PLAN	text
42	Buffers: shared hit=1
43	-> Hash (cost=1.04..1.04 rows=4 width=8) (actual time=0.010..0.011 rows=4.00 loops=1)
44	Buckets: 1024 Batches: 1 Memory Usage: 9kB
45	Buffers: shared hit=1
46	-> Seq Scan on productos prod (cost=0.00..1.04 rows=4 width=8) (actual time=0.006..0.007 rows=4.00 loops=1)
47	Buffers: shared hit=1
48	-> Hash (cost=10.90..10.90 rows=90 width=222) (actual time=0.038..0.038 rows=2.00 loops=1)
49	Buckets: 1024 Batches: 1 Memory Usage: 9kB
50	Buffers: shared hit=1
51	-> Seq Scan on sucursales suc (cost=0.00..10.90 rows=90 width=222) (actual time=0.026..0.026 rows=2.00 loops=1)
52	Buffers: shared hit=1
53	Planning:
54	Buffers: shared hit=47 read=2
55	Planning Time: 3.412 ms
56	Execution Time: 2.316 ms

OPTIMIZACIÓN AVANZADA: ÍNDICE COMPUESTO EN MOVIMIENTOS

Para concluir el esquema de optimización física con un criterio de alta escalabilidad, se analizó el comportamiento de los filtros condicionales indexando columnas combinadas. Se tomó como caso de estudio la **Consulta Avanzada N° 1 (Reporte de Recaudación Trimestral por Marca)**, debido a su dependencia crítica de un filtrado compuesto por tipo de operación y rango temporal dentro del WHERE.

FASE 1: Estado Inicial (EI ANTES)

- **Métrica de Rendimiento:** Execution Time: 1.007 ms.
- **Diagnóstico del Query Plan:** El planificador evalúa de forma lineal los filtros condicionales de la tabla, segmentando el descarte en pasos independientes.

- **Causa de la Demora:** El motor filtra de forma aislada las columnas del WHERE, duplicando los pasos lógicos de descarte sobre el historial completo.

```
168
169 --Índice Compuesto Avanzado en "Movimientos"
170 EXPLAIN ANALYZE
171 SELECT
172     mrc.nombre AS marca_deportiva,
173     SUM(dm.cantidad) AS total_prendas_vendidas,
174     SUM(dm.cantidad * dm.precio_unitario) AS recaudacion_total
175 FROM Movimientos mov
176 INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
177 INNER JOIN Producto_Variante pv ON dm.id_variante = pv.id_variante
178 INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
179 INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
180 WHERE mov.tipo_movimiento = 'Salida'
181     AND mov.fecha_hora >= CURRENT_DATE - INTERVAL '3 months'
182 GROUP BY mrc.id_marca, mrc.nombre
183 ORDER BY recaudacion_total DESC;
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 48 Page No: 1 of 1

QUERY PLAN
text
Seq Scan on productos prod (cost=0.00..1.04 rows=8 width=8) (actual time=0.008..0.009 rows=4.00 loops=1)
Buffers: shared hit=1
Hash Join (cost=15.53..15.53 rows=4 width=226) (actual time=0.053..0.053 rows=4.00 loops=1)
Hash Cond: (mrc.id_marca = prod.id_marca)
Seq Scan on marcas mrc (cost=0.00..13.20 rows=320 width=222) (actual time=0.025..0.025 rows=4.00 loops=1)
Buffers: shared hit=2
Hash Cond: (mrc.id_marca = prod.id_marca)
Seq Scan on productos prod (cost=0.00..1.04 rows=8 width=8) (actual time=0.008..0.009 rows=4.00 loops=1)
Buffers: shared hit=1
Planning:
Buffers: shared hit=12
Planning Time: 1.003 ms
Execution Time: 1.007 ms

FASE 2: Implementación Física (La Solución)

Se inyectó en la estructura del motor un índice compuesto multi-columna basado en un árbol B-Tree, unificando los criterios del tipo de movimiento y el orden cronológico de las operaciones en un único bloque físico:

```
186 -- Creacion de indices para optimizar el rendimiento de las consultas
187 CREATE INDEX idx_movimientos_tipo_fecha
188 ON public.Movimientos(tipo_movimiento, fecha_hora);
```

Data Output Mensajes Notificaciones

CREATE INDEX

Consulta retornó exitosamente en 112 msec.

FASE 3: Estado Optimizado (El DESPUÉS)

- **Métrica de Rendimiento:** Execution Time: 0.793 ms.
- **Diagnóstico del Query Plan:** El planificador consolida la reestructuración de los accesos físicos, aprovechando la persistencia de las páginas en los

búferes de memoria y los punteros ordenados del índice compuesto de doble columna.

- **Análisis del Resultado:** La optimización física definitiva redujo el tiempo de procesamiento de la consulta original a un valor de élite de 0.793 ms. Al unificar los criterios del tipo de movimiento y el rango cronológico en una sola estructura indexada, el motor minimiza las pasadas redundantes de evaluación en la CPU y los tiempos de planificación. Esto asegura que la consulta se resuelva de forma inmediata en la memoria intermedia, blindando el sistema contra la degradación de rendimiento cuando el volumen de operaciones escale a niveles masivos en producción.

169	--Índice Compuesto Avanzado en "Movimientos"
170	EXPLAIN ANALYZE
171	SELECT
172	mrc.nombre AS marca_deportiva,
173	SUM(dm.cantidad) AS total_prendas_vendidas,
174	SUM(dm.cantidad * dm.precio_unitario) AS recaudacion_total
175	FROM Movimientos mov
176	INNER JOIN Detalle_Movimientos dm ON mov.id_movimiento = dm.id_movimiento
177	INNER JOIN Producto_Variante pv ON dm.id_variante = pv.id_variante
178	INNER JOIN Productos prod ON pv.id_producto = prod.id_producto
179	INNER JOIN Marcas mrc ON prod.id_marca = mrc.id_marca
180	WHERE mov.tipo_movimiento = 'Salida'
181	AND mov.fecha_hora >= CURRENT_DATE - INTERVAL '3 months'
182	GROUP BY mrc.id_marca, mrc.nombre
183	ORDER BY recaudacion_total DESC;

Data Output	Mensajes	Notificaciones
Showing rows: 1 to 52 Page No: 1 of 1		
QUERY PLAN text		
35	Buffers: shared hit=1	
36	-> Hash (cost=15.53, 15.53 rows=4 width=226) (actual time=0.092..0.093 rows=4.00 loops=1)	
37	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
38	Buffers: shared hit=2	
39	-> Hash Join (cost=1.09..15.53 rows=4 width=226) (actual time=0.080..0.082 rows=4.00 loops=1)	
40	Hash Cond: (mrc.id_marca = prod.id_marca)	
41	Buffers: shared hit=2	
42	-> Seq Scan on marcas mrc (cost=0.00..13.20 rows=320 width=222) (actual time=0.040..0.040 rows=4.00 loops=1)	
43	Buffers: shared hit=1	
44	-> Hash (cost=1.04..1.04 rows=4 width=8) (actual time=0.015..0.015 rows=4.00 loops=1)	
45	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
46	Buffers: shared hit=1	
47	-> Seq Scan on productos prod (cost=0.00..1.04 rows=4 width=8) (actual time=0.010..0.011 rows=4.00 loops=1)	
48	Buffers: shared hit=1	
49	Planning:	
50	Buffers: shared hit=12	
51	Planning Time: 0.905 ms	
52	Execution Time: 0.793 ms	